Cada pregunta tiene cuatro posibles respuestas y solo una de ellas es correcta. Para cada pregunta, salvo que el enunciado de ésta indique explícitamente lo contrario, indique la respuesta VERDADERA.

1. **En la fase de diseño de una aplicación software para una tienda online se ha añadido una nueva clase al diagrama de clases de diseño cuya responsabilidad es gestionar la creación, edición y destrucción de objetos de la clase 'Producto' definida en análisis, a fin de permitir el alta, modificación y baja de productos en la aplicación. Además, cuando un usuario solicite dar de alta, editar o dar de baja un producto, esta nueva clase comprobará que el usuario tiene permisos para hacerlo. ¿De qué tipo es la nueva clase que se ha añadido al diagrama?**
   - **a.** Controlador.
   - **b.** Clase del sistema.
   - **c.** Clase de interfaz de usuario.
   - **d.** Clase del dominio.

2. **En la fase de diseño de una aplicación móvil para gestionar un dispositivo de reproducción de música se ha añadido una nueva clase al diagrama de clases de diseño cuya responsabilidad es permitir al usuario enviar comandos de voz para variar el volumen. ¿De qué tipo es la nueva clase que se ha añadido al diagrama?**
   - **a.** Controlador.
   - **b.** Clase de interfaz de usuario.
   - **c.** Clase del sistema.
   - **d.** Clase del dominio.

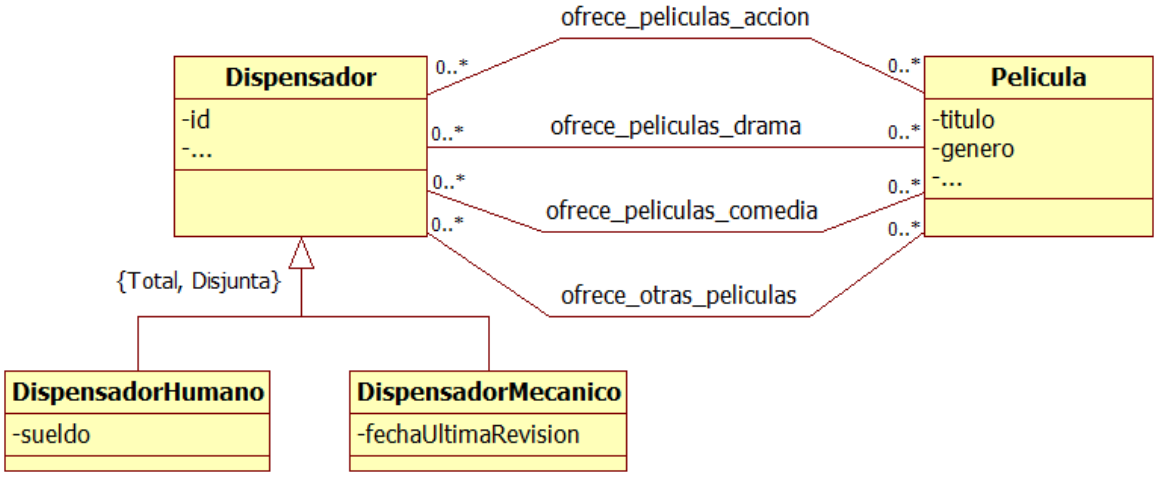
3. **Indique la afirmación verdadera:**
   - **a.** El acoplamiento de un sistema software puede definirse como la forma y grado de interdependencia entre sus módulos. Por lo tanto, todo sistema software bien diseñado debería tener un acoplamiento nulo.
   - **b.** La cohesión de un sistema software puede definirse como la forma y grado en que las tareas realizadas por cada módulo se relacionan entre sí. Por lo tanto, es deseable que un sistema software tenga una alta cohesión.
   - **c.** La modularidad de un sistema software puede definirse como el grado en que dicho sistema se compone de partes discretas denominadas módulos. Los sistemas software deberían ser modulares y tener la máxima cantidad de módulos posible.
   - **d.** La modularidad de un sistema software no tiene relación con su acoplamiento ni con su cohesión.

4. **Dado el siguiente fragmento de código, se puede afirmar que existe:**

```
public class Operacion {
  int a,b;
  public int m1() {
    a = 5;
    b = 9;
    return m2(a + b);
  }
  public int m2(int c) {
    if (c>10) {
      return a;
    } else {
      return b;
    }
  }
}
```

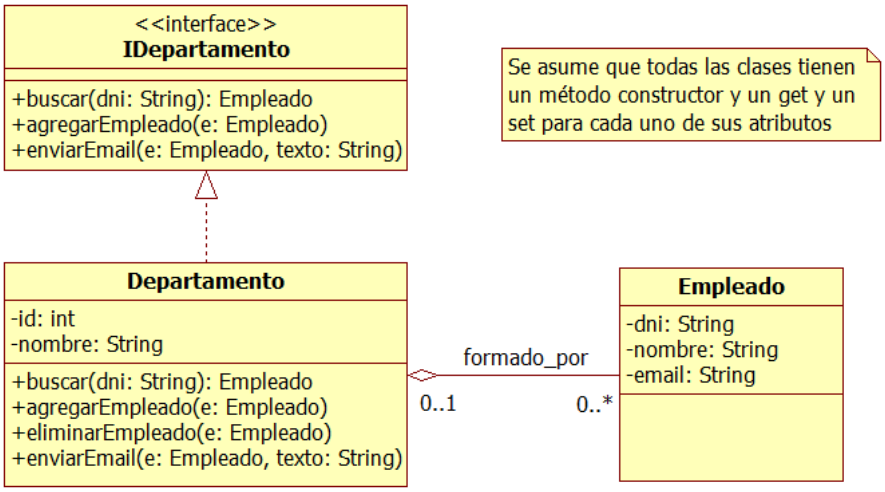
   - **a.** Un acoplamiento de contenido.
   - **b.** Un acoplamiento de control dañino, ya que un valor generado en el método m2 afecta al comportamiento del método m1.
   - **c.** Un acoplamiento de marcado, ya que las operaciones internas del método m1 marcan el comportamiento del método m2.
   - **d.** Un acoplamiento común, ya que los métodos m1 y m2 comparten memoria.

5. Dado el siguiente fragmento de un diagrama de clases de un sistema de gestión de videoclubs que se está modelando se puede afirmar que:



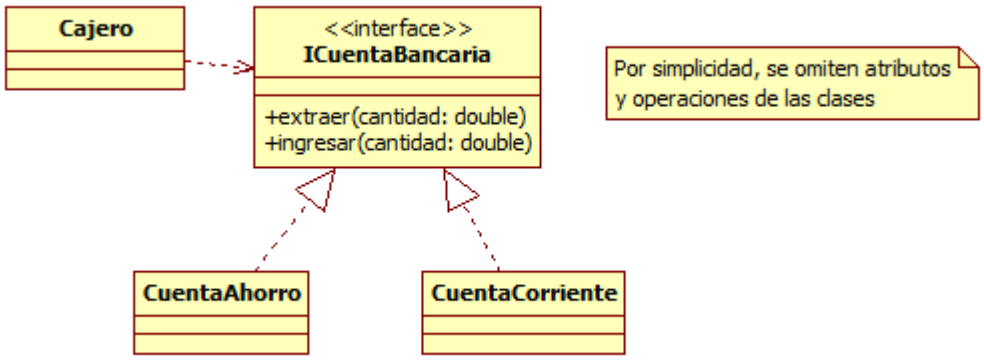
a. Se ha aplicado el principio de abstracción exclusivamente por especialización.
b. Se ha aplicado principio de abstracción exclusivamente por agrupación.
c. Se ha aplicado el principio de abstracción tanto por especialización como por agrupación.
d. No se ha aplicado el principio de abstracción ni por especialización ni por agrupación.

6. ¿Qué cambio habría que realizar para que el diseño representado por el siguiente diagrama de clases cumpliese con el principio de responsabilidad única?



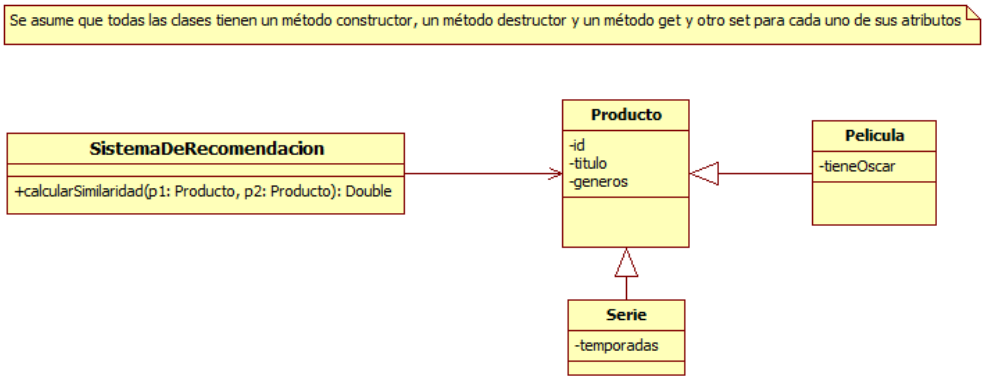
a. Aplicar el patrón de diseño Método factoría para que la creación de nuevos empleados se delegue a una nueva clase.
b. Crear una nueva clase y trasladar a ella la responsabilidad del envío de correos electrónicos.
c. Dividir la interfaz IDepartamento en interfaces más ligeras.
d. Ninguno, ya que el diseño cumple con el principio de responsabilidad única.

7. Un banco está realizando el diseño de su sistema de gestión, en concreto del módulo encargado de las cuentas bancarias. Actualmente el banco admite dos tipos de cuenta: cuenta de ahorro y cuenta corriente. Sin embargo, el banco prevé ofertar nuevos tipos de cuenta en el futuro, después de que el sistema haya sido implantado. ¿Qué principio de diseño han seguido para manejar esta situación, obteniendo el diseño que se muestra a continuación?



a. Principio de segregación de interfaces.
b. Principio de responsabilidad única.
c. Principio abierto/cerrado.
d. Ninguno.

8. Dado el siguiente diagrama de clases, se puede afirmar que el diseño representado:



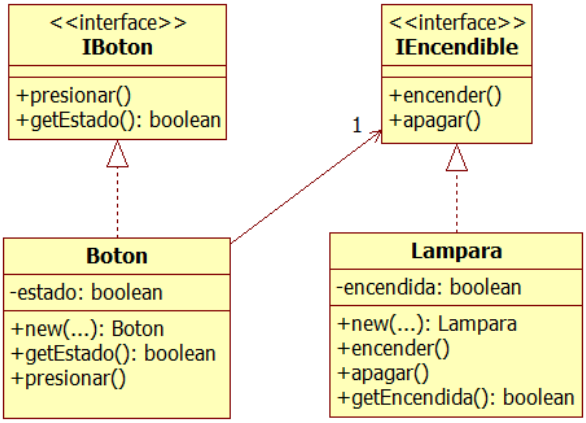

```
public double calcularSimilaridad(Producto p1, Producto p2) {
    int generosCoincidentes = 0;  String[] generosP1 = p1.getGeneros();  String[] generosP2 = p2.getGeneros();
    for(int i=0; i<generosP1.length; i++) {
        for(int j=0; j<generosP2.length; j++) {
            if(generosP1[i].equals(generosP2[j])){
                generosCoincidentes += 1; break;
            }
        }
    }
    return 2.0*generosCoincidentes/(generosP1.length + generosP2.length);
}
```

a. Cumple con el principio de sustitución de Liskov.
b. No cumple con el principio de sustitución de Liskov porque calcularSimilaridad no puede recibir como parámetros objetos de clases derivadas de Serie y/o Pelicula.
c. No cumple con el principio de sustitución de Liskov porque Serie y Pelicula tienen métodos públicos diferentes y por tanto no presentan una misma interfaz.
d. No cumple con el principio de sustitución de Liskov porque el sistema de recomendación necesita tener conocimiento sobre clases hijas de Producto.

9. Dado un diseño representado mediante un diagrama de clases, podemos afirmar que este diseño incumple el principio de segregación de interfaces siempre que alguna clase se ve obligada a:

- **a.** Implementar varias interfaces.
- **b.** Implementar una interfaz con todos sus métodos.
- **c.** Depender de varias interfaces.
- **d.** Depender innecesariamente de algún método provisto por una interfaz.

10. Dado el siguiente diagrama de clases y considerando que se ha aplicado el principio de inversión de dependencias, podemos afirmar que:

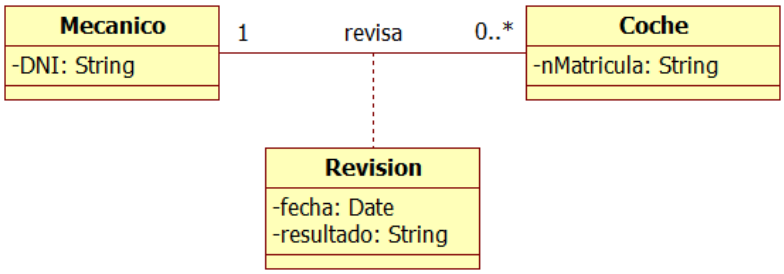


- **a.** El principio de inversión de dependencias no se ha aplicado correctamente porque la clase Boton depende de los detalles de implementación de la clase Lampara.
- **b.** El principio de inversión de dependencias no se ha aplicado correctamente porque la clase Boton no puede acceder a todos los métodos de la clase Lampara.
- **c.** Si se añadiese una nueva clase Motor que implementase la interfaz IEncendible, ésta podría ser utilizada por la clase Boton sin necesidad de modificar el código de la clase Boton.
- **d.** La clase Boton proporciona una interfaz para desacoplar su código del de la clase Lampara.

11. La aplicación de los principios SOLID permite mitigar ciertos defectos en los diseños, de forma que:

- **a.** Al aplicar el principio de responsabilidad única se reduce el acoplamiento.
- **b.** Al aplicar el principio de inversión de dependencias se aumenta la cohesión.
- **c.** Al aplicar el principio de segregación de interfaces se puede lograr aumentar la cohesión, pero nunca reducir el acoplamiento.
- **d.** Ninguna de las otras opciones es verdadera.

12. Se ha implementado una aplicación empleando el lenguaje de programación Java de acuerdo al diagrama de clases adjunto. Teniendo en cuenta este diagrama, ¿cuál de los siguientes atributos debería estar en la clase "Mecanico"?

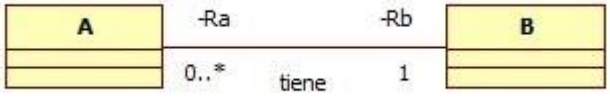


- **a.** private Revision revision.
- **b.** private List<Revision> revisiones.
- **c.** private Coche coche.
- **d.** private List<Coche> coches.

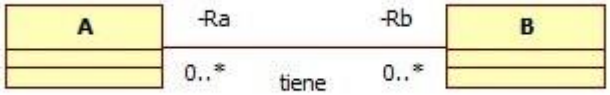
13. Dado el siguiente esqueleto de código Java correspondiente a la implementación de dos clases A y B:

```
public class A {
  private List<B> Rb;
}
public class B {
  private A Ra;
}
```

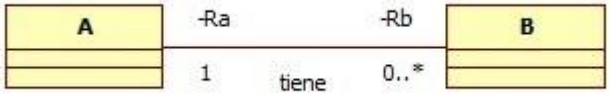
¿Cuál de las siguientes relaciones podría ser la que había entre las clases A y B en el diagrama de clases?



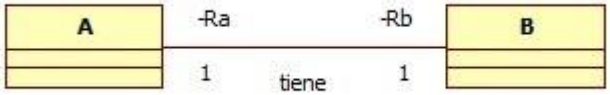
a.



b.

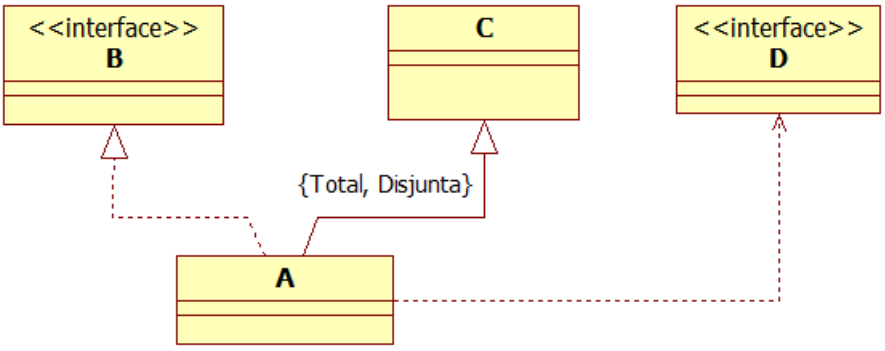


c.



d.

14. Dado el siguiente diagrama de clases en el que por simplicidad no se representan métodos ni atributos, ¿cuál debería ser la cabecera de la clase A implementada en Java?



a. public class A extends C implements D.
b. public class A extends C implements B, D.
c. public class A extends C implements B.
d. public class A extends B implements C.

15. Una aplicación web cliente integrada en un portal de comercio electrónico muestra a los usuarios recomendaciones de productos relacionados con el que están viendo en ese momento. Para ello, la aplicación realiza peticiones a un servidor cuando los usuarios seleccionan un nuevo producto. Sin embargo, esta aplicación almacena todas las recomendaciones realizadas de modo que, si los usuarios vuelven a visitar un mismo producto, en lugar de realizar una nueva petición al servidor, muestra las recomendaciones que había almacenado previamente. ¿Qué patrón de diseño es adecuado para implementar esta funcionalidad?

a. Patrón Proxy.
b. Patrón Método factoría.
c. Patrón Singleton.
d. Patrón MVC.

16. Para desarrollar interfaces gráficas en Android se hace uso de objetos Java pertenecientes a la clase android.view.View o a clases derivadas de esta. A estos objetos se pueden registrar escuchadores (listeners), los cuales permiten especificar qué código se debe ejecutar cuando ocurre un determinado evento asíncrono, por ejemplo, cuando el usuario pulsa sobre un botón. En base a esta información, ¿cuál es el patrón de diseño utilizado para gestionar estos eventos?

a. Sense-Compute-Control.
b. Publish-Subscribe.
c. Comando.
d. Observer.

17. Dado el siguiente fragmento de código Java correspondiente a una clase implementada de acuerdo al patrón de diseño Singleton, se puede afirmar que:

```
import java.util.List;

public class Agenda {
  private static final Agenda agenda = new Agenda();
  private List<Contacto> contactos;

  public Agenda(){}

  public static Agenda getInstancia(){
    return agenda;
  }
  [...]
}
```

a. El patrón Singleton se ha implementado correctamente.
b. El patrón Singleton no se ha implementado correctamente, ya que al ser el constructor público no se garantiza que exista una única instancia de la clase Agenda.
c. La correcta aplicación del patrón Singleton requiere emplear varias clases relacionadas entre sí, de modo que la implementación mostrada es errónea.
d. No es adecuado aplicar el patrón Singleton para esta situación, ya que es igual de beneficioso emplear una variable global.

18. Para desarrollar un sistema de gestión bibliotecaria se modelan las clases Catalogo y Recurso. La clase abstracta Recurso tiene varias clases hijas: Libro, AudioLibro, y Audiovisual. La clase Catalogo debe crear los Recursos, pero se pretende flexibilizar este proceso de creación todo lo posible por lo que lo más conveniente es:

a. Implementar el patrón Método factoría, para lo cual habría que crear una nueva clase con un método factoría responsable de crear los objetos convenientes de las clases hijas de Recurso.
b. Implementar el patrón Método factoría, creando en la clase Catalogo un método factoría encargado de crear los objetos convenientes de las clases hijas de Recurso.
c. Aunque sería posible implementar el patrón Método factoría, éste no sería útil porque tendría que existir una clase CatalogoConcreto que estaría altamente acoplada con la clase Recurso.
d. No se puede implementar el patrón Método factoría.

19. El patrón de diseño Adaptador o Adapter:

- **a.** Se puede emplear para transformar una interfaz provista por una clase en otra interfaz esperada por otras clases.
- **b.** Se puede emplear para solicitar una operación a un objeto sin conocer el contenido de esta operación ni el receptor real de la misma.
- **c.** Se considera un patrón de comportamiento, ya que adapta la forma en la que se comporta una determinada clase para lograr que pueda emplear una interfaz que antes no podía.
- **d.** Solo se puede aplicar empleando herencia.

20. Dada la siguiente clase enmarcada en un sistema de gestión que será utilizado por una discográfica, se puede afirmar que:

- **a.** No se puede identificar ningún antipatrón.
- **b.** Es posible identificar el antipatrón Código Spagetti, ya que se puede determinar que la programación de los métodos tiene una estructura muy compleja.
- **c.** Se puede identificar el antipatrón God Class ya que, aunque se cumple el principio de responsabilidad única, la clase Discográfica concentra una gran cantidad de atributos y métodos.
- **d.** Se puede identificar el antipatrón God Class, siendo posible mejorar el diseño creando dos clases, Artista y Disco, que alberguen los atributos y métodos relacionados con esas entidades.

21. Se pretende realizar una aplicación de escritorio sencilla en Java utilizando la librería gráfica Swing para la implementación de la interfaz gráfica de usuario. Los componentes Swing atenderán los eventos ocasionados por la interacción del usuario con la aplicación, los cuáles serán posteriormente procesados por un componente de negocio que a su vez podrá utilizar otro componente para almacenar o recuperar datos. ¿Qué patrón o estilo arquitectónico se está aplicando?

- **a.** Pipe and Filter.
- **b.** State-Logic-Display.
- **c.** Sense-Compute-Control.
- **d.** Ninguna de las otras opciones es correcta.

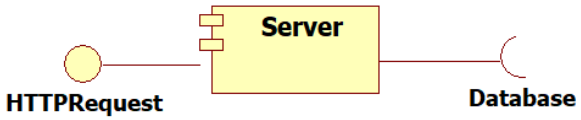
22. Se está desarrollando un portal de compras en línea, al cual deberá de poder accederse empleando distintos tipos de dispositivos mediante un navegador web. Por tanto, se desea desacoplar tanto como sea posible la representación gráfica de la información de la forma en la que ésta se almacena y estructura dentro del sistema. ¿Qué patrón o estilo arquitectónico se adapta mejor a estos requisitos?

- **a.** El patrón arquitectónico Sense-Compute-Control.
- **b.** El estilo arquitectónico Peer-to-peer.
- **c.** El patrón arquitectónico Modelo-Vista-Controlador.
- **d.** El estilo arquitectónico Pipe and Filter.

23. Una compañía quiere desarrollar un sistema software para permitir a jugadores profesionales de e-Sports transmitir sus partidas en directo, de tal manera que, a través de una plataforma web, cualquier aficionado pueda elegir el conjunto de partidas que quiere ver en un momento determinado (cada aficionado recibirá 1 flujo de vídeo por partida elegida). ¿Qué estilo arquitectónico se debería utilizar para desarrollar este sistema software a fin de conseguir una difusión eficiente de los flujos de vídeo enviados a los espectadores?

a. Peer to peer.
b. Pipe and Filter.
c. Event-Based.
d. Publish-Subscribe.

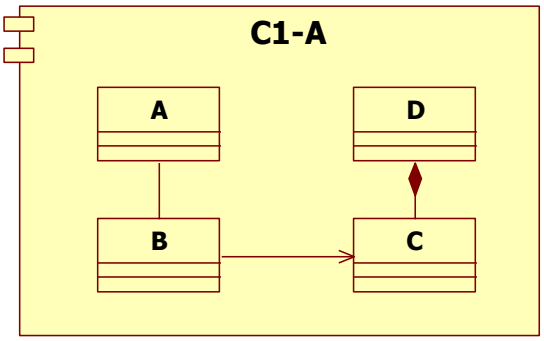
24. De acuerdo a la siguiente figura, podemos afirmar que el componente Server representado:



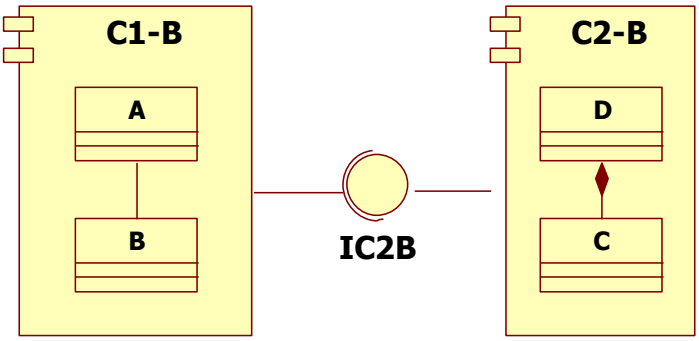
a. Contiene la implementación de una única clase.
b. Se podría sustituir por otro componente que implementase la interfaz HTTPRequest.
c. Provee la interfaz Database y requiere la interfaz HTTPRequest.
d. No está bien diseñado puesto que no encapsula adecuadamente su implementación.

25. Cada uno de los diagramas de componentes adjuntos (los cuales indican las clases pertenecientes a cada componente) representa un posible diseño para un sistema software que está compuesto por un total de 4 clases: A, B, C y D. El diagrama de componentes situado a la izquierda muestra el diseño A y el que está situado a la derecha muestra el diseño B. En base a estos diagramas de componentes, se puede afirmar que:

Diseño A

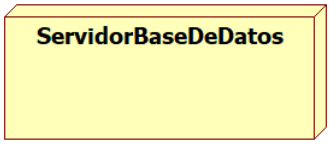


Diseño B



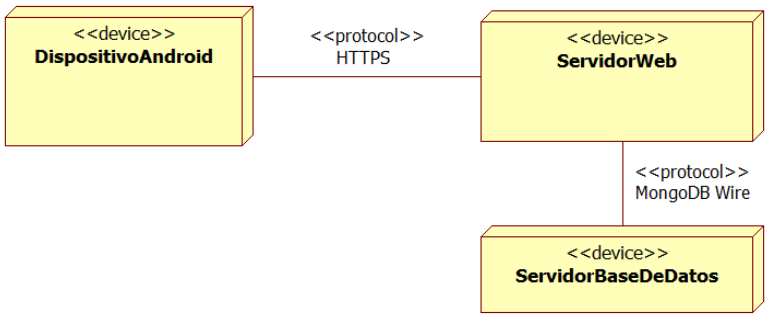
a. El diseño A es mejor que el B en términos de acoplamiento.
b. El diseño A es mejor que el B en términos de cohesión.
c. El diseño A es mejor que el B en términos de modularidad.
d. Ninguna de las otras opciones es verdadera.

26. De acuerdo al Modelo de Vistas de Arquitectura 4+1, ¿para representar qué vista se podría utilizar un diagrama UML que incluyese el siguiente elemento?



a. Vista lógica.
b. Vista física.
c. Vista de desarrollo.
d. El diagrama UML no sería útil para representar ninguna de las vistas definidas por el Modelo de Vistas de Arquitectura 4+1.

27. Dado el siguiente diagrama de despliegue correspondiente a una aplicación Android, es posible afirmar con total seguridad que:



a. Al menos uno de los componentes de la aplicación es utilizado por el dispositivo Android del usuario final.
b. El diagrama de despliegue de la aplicación representa un total de 2 nodos.
c. La aplicación tiene un total de tres componentes.
d. La aplicación tiene un total de dos interfaces entre componentes.